

Sadia Alam

CS441: Programming Language & Design

03/29/2026

AI generated codes

Stripping comments:

```
</> racket

(define (strip-comments str)
  (define chars (string->list str))

  (define (helper lst in-comment?)
    (cond
      ;; End of input
      [(null? lst)
       (if in-comment?
           (error "Unterminated comment")
           '())]

      ;; Inside comment
      [in-comment?
       (if (and (pair? (cdr lst))
                (char=? (car lst) #\*)
                (char=? (car (cdr lst)) #\/))
           ;; End comment
           (helper (cdr (cdr lst)) #f)
           ;; Skip char
           (helper (cdr lst) #t)))]

      ;; Outside comment
      [else
       (if (and (pair? (cdr lst))
                (char=? (car lst) #\/)
                (char=? (car (cdr lst)) #\*))
           ;; Start comment
           (cons #\space (helper (cdr (cdr lst)) #t))
           ;; Keep char
           (cons (car lst) (helper (cdr lst) #f)))))]

    (list->string (helper chars #f)))
```

I modified structure and implementation details of the code. Instead of nested helper my program uses let loop and uses syntactic sugar. My program doesn't insert space in replacement of comments. Also, the result list is differently implemented.

```
(define (tokenize str)
  (define len (string-length str))

  ;; -----
  ;; character helpers
  ;; -----

  (define (at-end? i)
    (>= i len))

  (define (current-char i)
    (string-ref str i))

  (define (has-next? i)
    (< (+ i 1) len))

  (define (next-char i)
    (string-ref str (+ i 1)))

  (define (whitespace? ch)
    (char-whitespace? ch))

  (define (leading-char? ch)
    (or (char-alphabetic? ch)
        (char=? ch #\_)))

  (define (id-tail-char? ch)
    (or (char-alphabetic? ch)
        (char-numeric? ch)
        (char=? ch #\_))
    (char=? ch #\_-)))

  (define (digit? ch)
    (char-numeric? ch))

  .. -----
```

```
(define (scan i tokens prev-type)
  (cond
    ;; end of input
    [(at-end? i)
     (reverse tokens)]

    ;; skip whitespace
    [(whitespace? (current-char i))
     (scan (+ i 1) tokens prev-type)]

    ;; :=
    [(and (has-next? i)
          (char=? (current-char i) #\)
           (char=? (next-char i) #\=))
     (scan (+ i 2)
            (cons (token 'ASSIGN "!=") tokens)
            'ASSIGN)]

    ;; !=
    [(and (has-next? i)
          (char=? (current-char i) #\)
           (char=? (next-char i) #\<))
     (scan (+ i 2)
            (cons (token 'NEQ "!=") tokens)
            'NEQ)]

    ;; bare ! is invalid
    [(char=? (current-char i) #\)
     (error "Lexical Error: ! is not a valid token unless followed by =")]

    ;; >=
    [(and (has-next? i)
          (char=? (current-char i) #\)
           (char=? (next-char i) #\>))
     (scan (+ i 2)
            (cons (token 'GE ">=") tokens)
            'GE)]

    ;; <=
    [(and (has-next? i)
          (char=? (current-char i) #\)
           (char=? (next-char i) #\<))
     (scan (+ i 2)
            (cons (token 'LE "<=") tokens)
            'LE)]
```

```

(char=? (next-char 1) #\=))
(scan (+ i 2)
  (cons (token 'LE "<=") tokens)
  'LE))

;; identifier / keyword
[(leading-char? (current-char i))
 (define result (read-identifier i))
 (define tok (car result))
 (define next-i (cdr result))
 (scan next-i
  (cons tok tokens)
  (token-type tok))]

;; number beginning with digit
[(digit? (current-char i))
 (define result (read-number i))
 (define tok (car result))
 (define next-i (cdr result))
 (scan next-i
  (cons tok tokens)
  (token-type tok))]

```

```

(char=? (current-char i) #\*)
(scan (+ i 1)
  (cons (token 'MULT "*" ) tokens)
  'MULT))

;; /
[(char=? (current-char i) #\/)
 (scan (+ i 1)
  (cons (token 'DIV "/" ) tokens)
  'DIV))

;; =
[(char=? (current-char i) #\=)
 (scan (+ i 1)
  (cons (token 'EQ "=") tokens)
  'EQ))

;; >
[(char=? (current-char i) #\>)
 (scan (+ i 1)
  (cons (token 'GT ">") tokens)
  'GT))

```

```

;; <
[(char=? (current-char i) #\<)
 (scan (+ i 1)
  (cons (token 'LT "<") tokens)
  'LT))

;; (
[(char=? (current-char i) #\()
 (scan (+ i 1)
  (cons (token 'LPAREN "(") tokens)
  'LPAREN))

;; )
[(char=? (current-char i) #\))
 (scan (+ i 1)
  (cons (token 'RPAREN ")") tokens)
  'RPAREN))

;; ;
[(char=? (current-char i) #\;)
 (scan (+ i 1)

```

```

;; =====
;; bare : is invalid
[(char=? (current-char i) #\:)
 (error "Lexical Error: expected :=, but found bare :")]

;; anything else is invalid
[else
 (error (format "Lexical Error: unexpected character ~a" (current-char i))))])

scan 0 '() 'START))

```

I've modified the structure of the scanning loop. My program uses separate helper functions for reading identifiers and keywords. Also, uses helpers and modified logic to read integers and floating points and enforcing their rules like no leading zeros and multiple decimal points. My program adds some more error handling cases such as unclosed comments, using '!'.

Parser:

```
;; =====
```

```

(define (peek tokens)
  (if (null? tokens)
      'EOF
      (car tokens)))

```

```

(define (consume tokens expected-type)
  (cond
    [(null? tokens)
     (error (format "Syntax Error: expected ~a, found EOF" expected-type))]
    [(eq? (token-type (car tokens)) expected-type)
     (cdr tokens)]
    [else
     (error
      (format "Syntax Error: expected ~a, found ~a"
              expected-type
              (token-type (car tokens))))]))

```

The peek & consume functions remained the same. But I added a separate helper function to get the type of the peeked character.

```

;; =====
;; Statement dispatcher
;; <Statement> ::= <AssignStmt> | <IfStmt> | <WhileStmt> | <PrintStmt>
;; =====

(define (parse-statement tokens)
  (case (token-type-safe (peek tokens))
    [(ID) (parse-assign-stmt tokens)]
    [(IF) (parse-if-stmt tokens)]
    [(WHILE) (parse-while-stmt tokens)]
    [(PRINT) (parse-print-stmt tokens)]
    [else
     (error
      (format "Syntax Error: expected statement, found ~a"
              (token-type-safe (peek tokens))))]))

```

This function remained the same

```

(define (parse-print-stmt tokens)
  (define tokens1 (consume tokens 'PRINT))

  (define expr-result (parse-expression tokens1))
  (define expr-ast (car expr-result))
  (define tokens2 (cdr expr-result))

  (define tokens3 (consume tokens2 'SEMI))

  (cons `(print ,expr-ast) tokens3))

```

```

(define (parse-if-stmt tokens)
  (define tokens1 (consume tokens 'IF))

  (define cmp-result (parse-comparison tokens1))
  (define cmp-ast (car cmp-result))
  (define tokens2 (cdr cmp-result))

  (define tokens3 (consume tokens2 'THEN))

  (define then-result (parse-stmt-list tokens3))
  (define then-stmts (car then-result))
  (define tokens4 (cdr then-result))

  (define tokens5 (consume tokens4 'ELSE))

  (define else-result (parse-stmt-list tokens5))
  (define else-stmts (car else-result))
  (define tokens6 (cdr else-result))

  (define tokens7 (consume tokens6 'END))

```

```

(define (parse-while-stmt tokens)
  (define tokens1 (consume tokens 'WHILE))

  (define cmp-result (parse-comparison tokens1))
  (define cmp-ast (car cmp-result))
  (define tokens2 (cdr cmp-result))

  (define tokens3 (consume tokens2 'DO))

  (define body-result (parse-stmt-list tokens3))
  (define body-stmts (car body-result))
  (define tokens4 (cdr body-result))

  (define tokens5 (consume tokens4 'END))

  (cons `(while ,cmp-ast ,body-stmts) tokens5))

```

I've modified the structure of the AST and returns.

```
(define (parse-expression tokens)
  (define first-result (parse-term tokens))
  (define first-ast (car first-result))
  (define remaining1 (cdr first-result))

  (define (loop left-ast toks)
    (define tok (peek toks))
    (cond
      [(eq? (token-type-safe tok) 'PLUS)
       (define right-result (parse-term (cdr toks)))
       (define right-ast (car right-result))
       (define remaining (cdr right-result))
       (loop `(+ ,left-ast ,right-ast) remaining)]

      [(eq? (token-type-safe tok) 'MINUS)
       (define right-result (parse-term (cdr toks)))
       (define right-ast (car right-result))
       (define remaining (cdr right-result))
       (loop `(- ,left-ast ,right-ast) remaining)]

      [else
```

```
        [else
         (cons left-ast toks)]))
  (loop first-ast remaining1))
```

```
(define (parse-term tokens)
  (define first-result (parse-factor tokens))
  (define first-ast (car first-result))
  (define remaining1 (cdr first-result))

  (define (loop left-ast toks)
    (define tok (peek toks))
    (cond
      [(eq? (token-type-safe tok) 'MULT)
       (define right-result (parse-factor (cdr toks)))
       (define right-ast (car right-result))
       (define remaining (cdr right-result))
       (loop `(* ,left-ast ,right-ast) remaining)]

      [(eq? (token-type-safe tok) 'DIV)
       (define right-result (parse-factor (cdr toks)))
       (define right-ast (car right-result))
       (define remaining (cdr right-result))
       (loop `(/ ,left-ast ,right-ast) remaining)]
```

```
        (define remaining (cdr right-result))
        (loop `(/ ,left-ast ,right-ast) remaining)]

      [else
       (cons left-ast toks)]))
  (loop first-ast remaining1))
```

```

(define (parse-factor tokens)
  (define tok (peek tokens))

  (case (token-type-safe tok)
    [(ID)
     (cons (string->symbol (token-value tok))
           (cdr tokens))]

    [(INT)
     (cons (token-value tok)
           (cdr tokens))]

    [(FP)
     (cons (token-value tok)
           (cdr tokens))]

    [(LPAREN)
     (define expr-result (parse-expression (cdr tokens)))
     (define expr-ast (car expr-result))
     (define remaining (cdr expr-result))
     (define after-rparen (consume remaining 'RPAREN))
     (cons expr-ast after-rparen)]

```

```

[else
 (error
  (format "Syntax Error: expected factor, found ~a"
         (token-type-safe tok)))))]

```

My program has modified returns for these above functions, uses helper function to check tokens and consumes the operators. The functions are more simplified in my program.